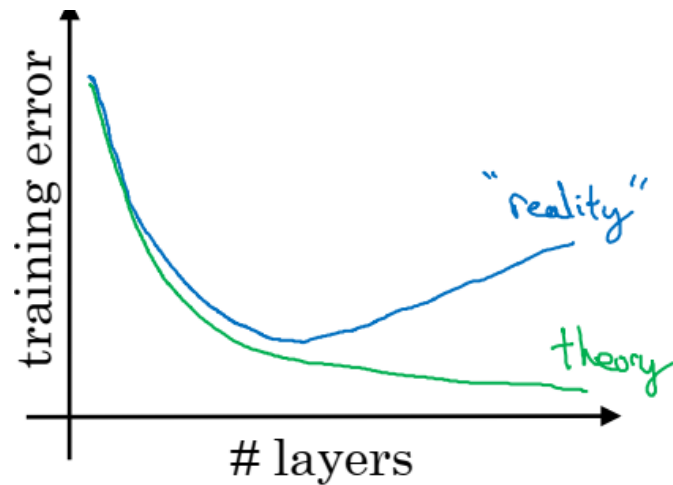


Deep Learning

RESNET

Deep vs Shallow Networks

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



-> The deeper model performs worse, but it's not caused by overfitting!

Deeper models are harder to optimize

- The deeper model should be able to perform at least as well as the shallower model.
- A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.

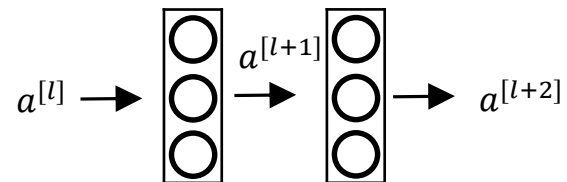
Challenges

- Deeper Neural Networks start to degrade in performance.
- Vanish/Exploding Gradient – May lead for extremely complex parameters initializations to make it work. Still might suffer from Vanish/Exploding even for the best parameters.
- Long training times – Due to too many training parameters.

Partial Solutions for Vanish/Exploding gradients

- Batch Normalization – To rescale the weights over some batch.
- Smart Initialization of weights – Like for example Xavier initialization.
- Train portions of the network individually.

Plain Network



$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

“linear”

$$a^{[l+1]} = g(z^{[l+1]})$$

“relu”

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

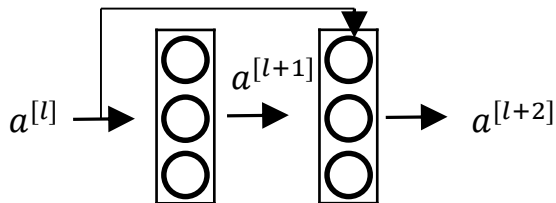
“output”

$$a^{[l+2]} = g(z^{[l+2]})$$

“relu on

output”

Residual Blocks



$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

“linear”

$$a^{[l+1]} = g(z^{[l+1]})$$

“relu”

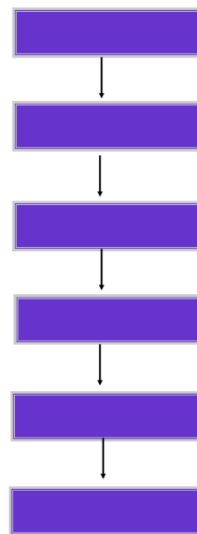
$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

“output”

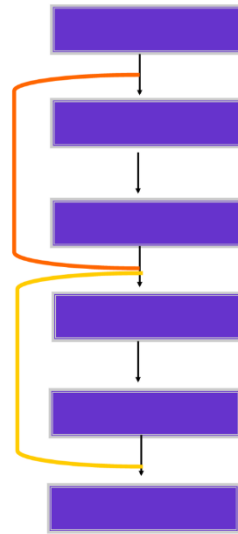
$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

“relu on output **plus input**”

PLAIN CONNECTIONS



RESIDUAL CONNECTIONS



Why Does it Work Well?

- Having a “regular” network that is very deep might actually hurt performance because of the vanishing and exploding gradients
- In most cases, ResNets will simply stop improving rather than decrease in performance
- $a^{[l+2]} = g(z^{[l+2]} + a^{[l]}) = g(w^{[l+1]}a^{[l+1]} + b^{[l]} + a^{[l]})$
- If the layer is not “useful”, L2 regularization will bring it's parameters very close to zero, resulting in $a^{[l+2]} = g(a^{[l]}) = a^{[l]}$ (when using ReLU)

Why Does it Work Well? (Cont)

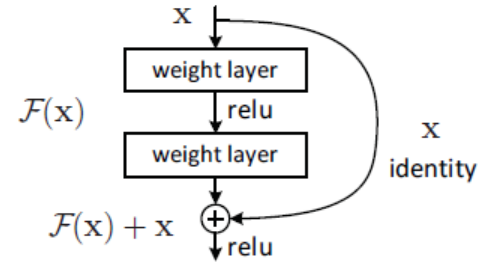
- In theory ResNet is still identical to plain networks, but in practice due to the above the convergence is much faster.
- No additional training parameters introduced.
- No additional complexity introduced.

ResNet

- A specialized network introduced by Microsoft.
- Connects inputs of layers into farther part of that network to allow “shortcuts”.
- Simple idea – great improvements with both performance and train time.

Skip Connections “shortcuts”

- Such connections are referred as skipped connections or shortcuts. In general similar models could skip over several layers.
- They refer to residual part of the network as a unit with input and output.
- Such residual part receives the input as an amplifier to its output – The dimensions usually are the same.
- no additional training parameters are used.



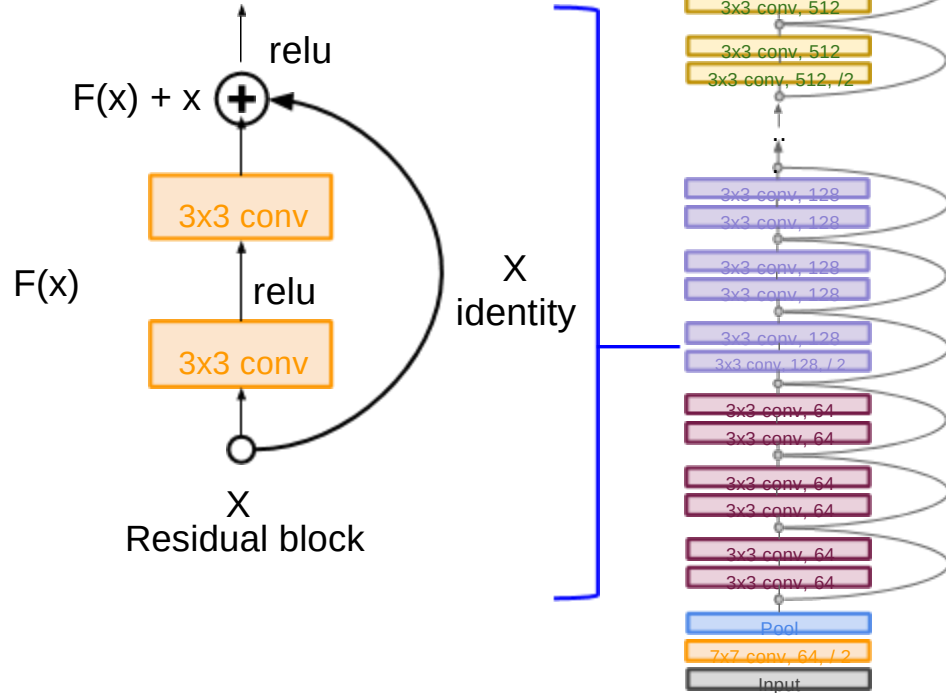
ResNet as a ConvNet

- Till now we talked about fully connected layers.
- The **ResNet** idea could easily expended into convolutional model.
- Other adaptations of this idea could be easily introduced to almost any kind of deep layered network.

ResNet Architecture

Full ResNet architecture:

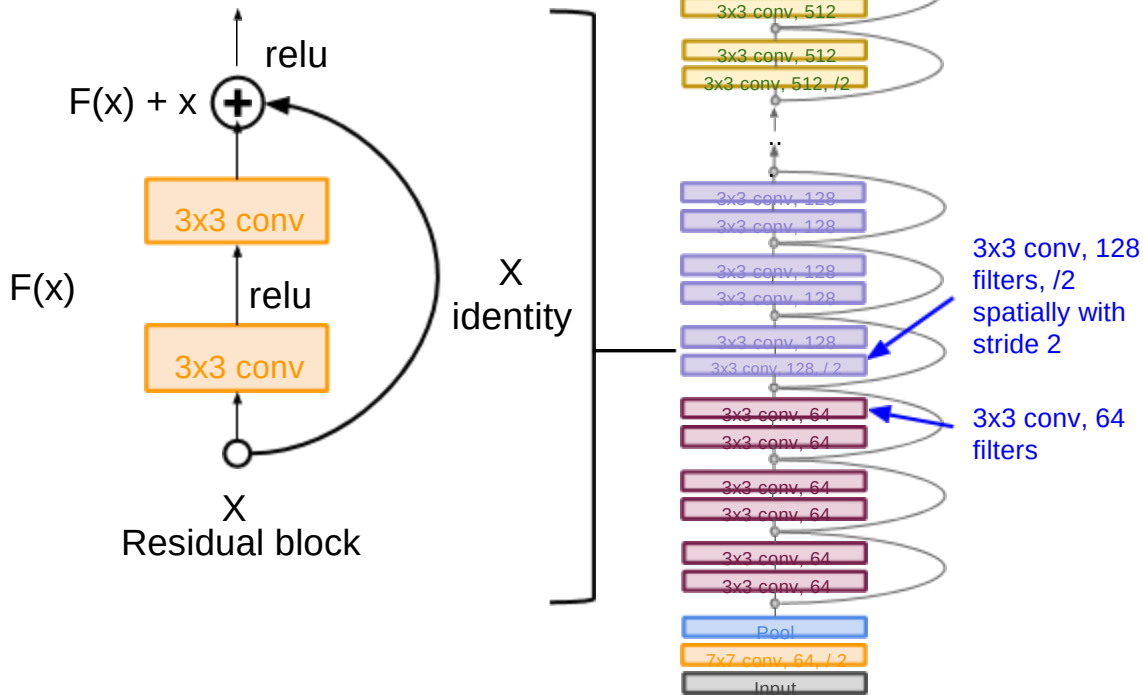
- Stack residual blocks
- Every residual block has two 3x3 conv layers



ResNet Architecture

Full ResNet architecture:

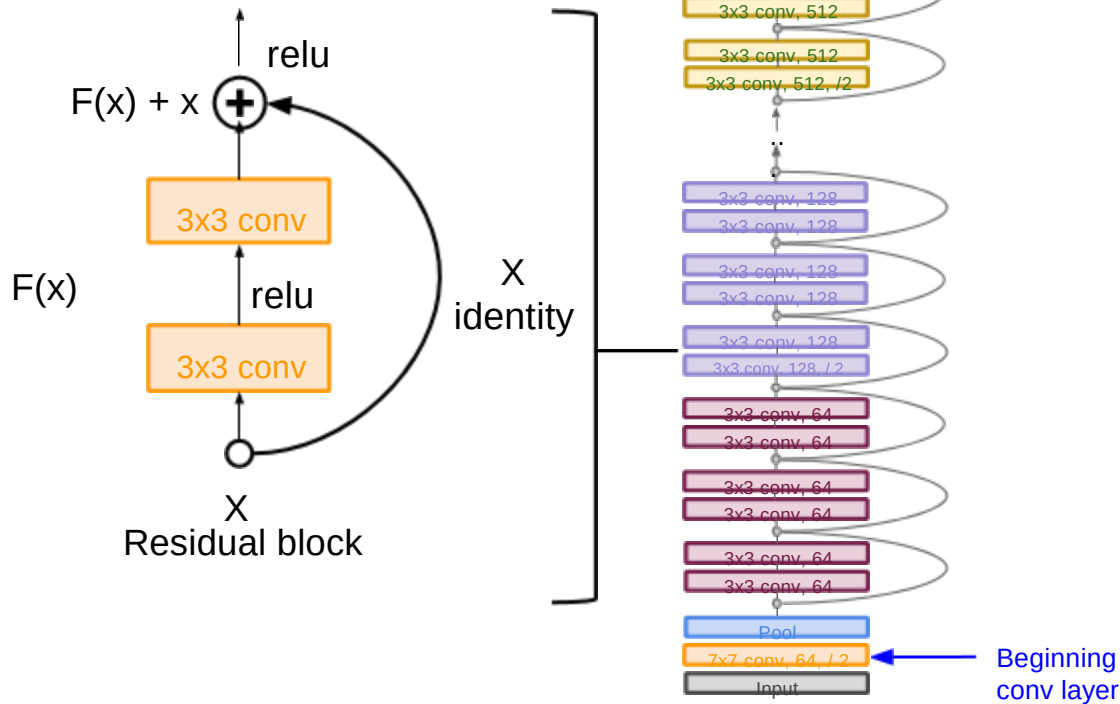
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)



ResNet Architecture

Full ResNet architecture:

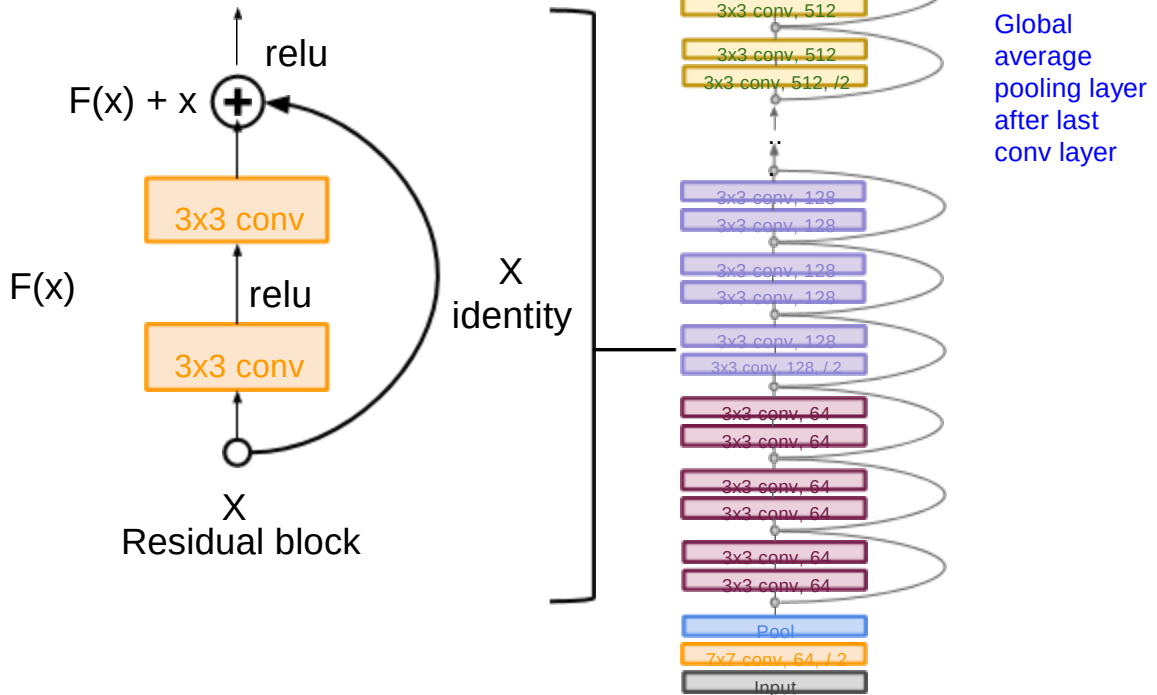
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning



ResNet Architecture

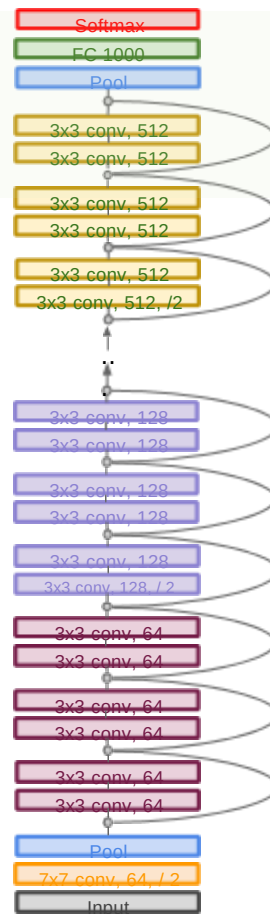
Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 ($/2$ in each dimension)
- Additional conv layer at the beginning
- No FC layers at the end (only FC 1000 to output classes)



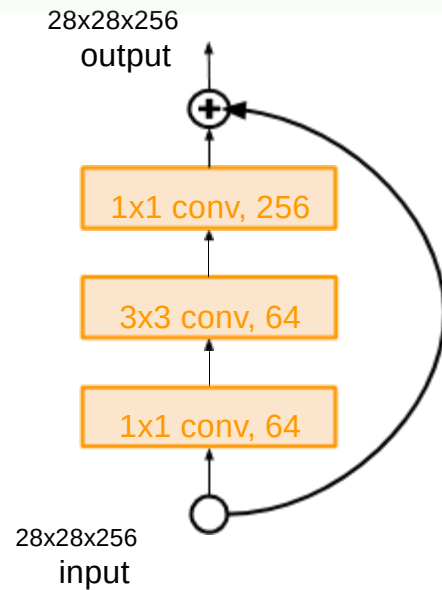
ResNet Architecture

Total depths of 34, 50, 101, or 152 layers for ImageNet



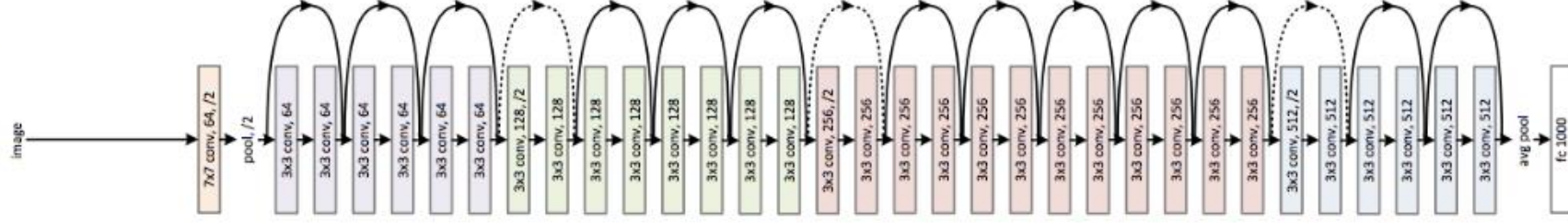
ResNet Architecture

For deeper networks
(ResNet-50+), use “bottleneck”
layer to improve efficiency
(similar to GoogLeNet)



Residual Blocks (skip connections)

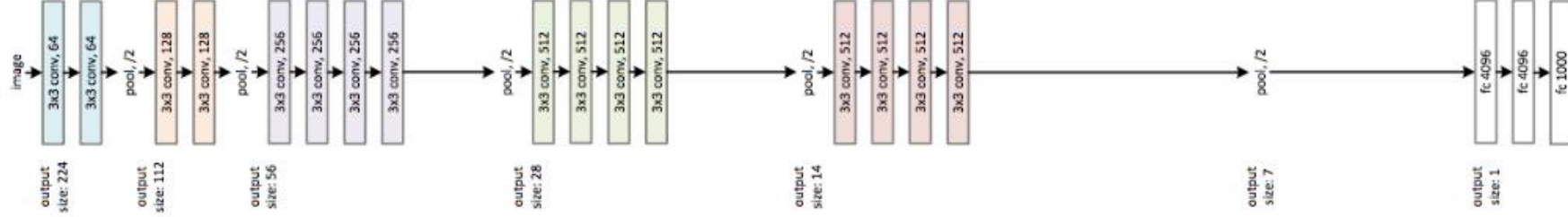
34-layer residual



34-layer plain



VGG-19



Training ResNet in practice

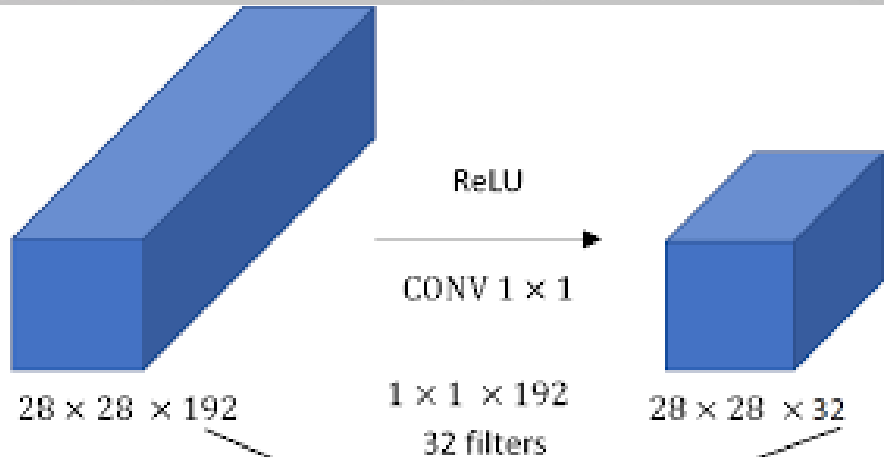
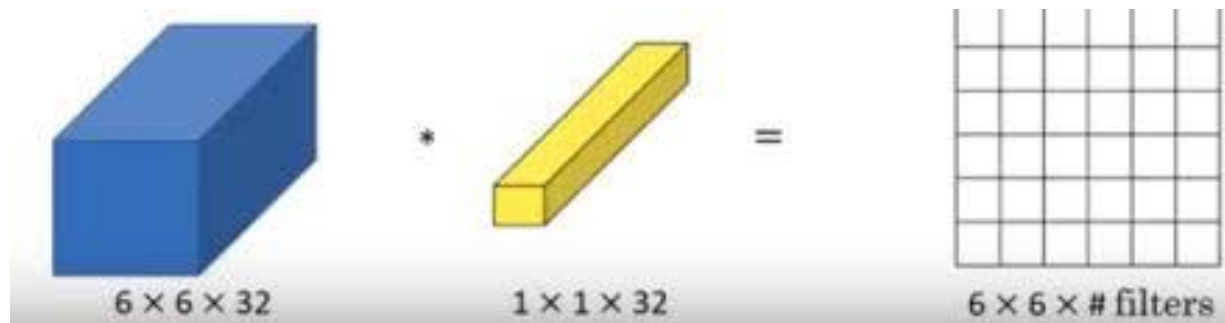
- Batch Normalization after every CONV layer.
- Xavier/2 initialization from He et al.
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus.
- Mini-batch size 256.
- Weight decay of $1e-5$.
- No dropout used.

GOOGLENET

GoogLeNet

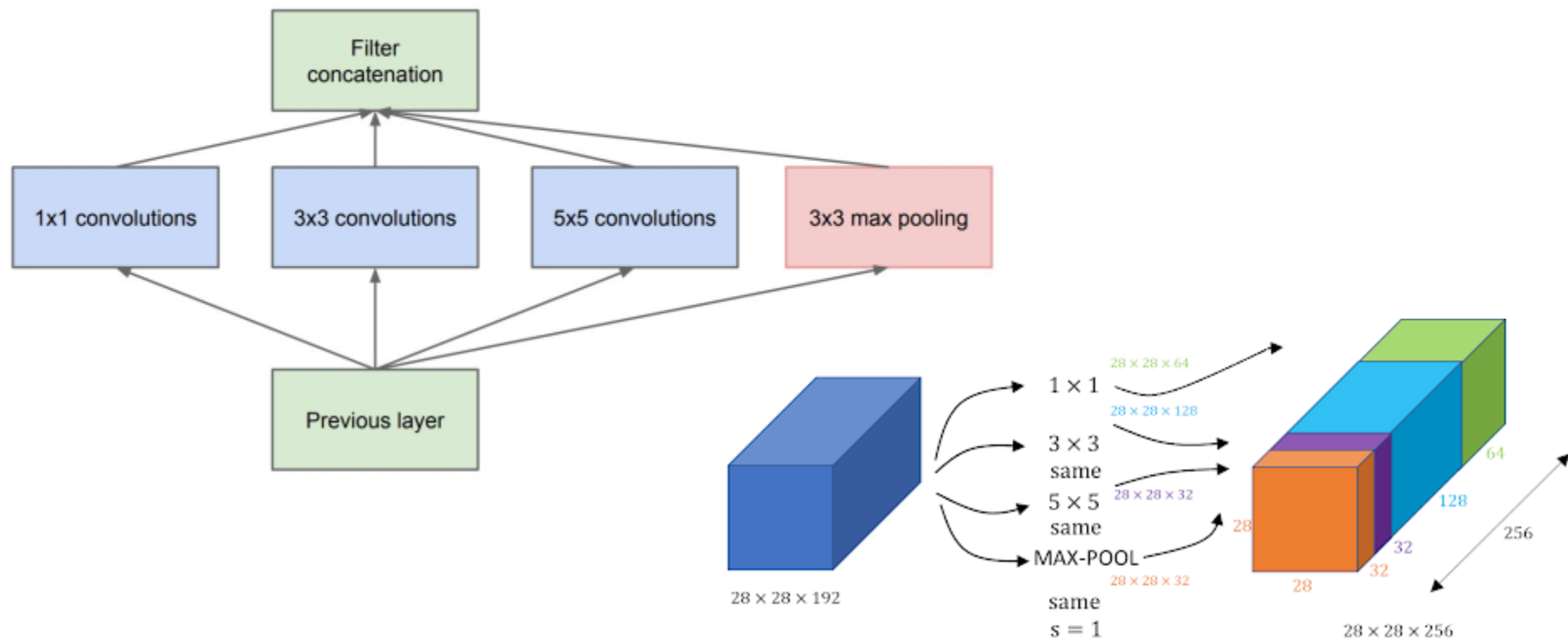
- **GoogLeNet.** The ILSVRC 2014 winner was a Convolutional Network from [Szegedy et al.](#) from Google. Its main contribution was the development of an *Inception Module* that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M). Additionally, this paper uses Average Pooling instead of Fully Connected layers at the top of the ConvNet, eliminating a large amount of parameters that do not seem to matter much. There are also several followup versions to the GoogLeNet.

1 X 1 Convolution

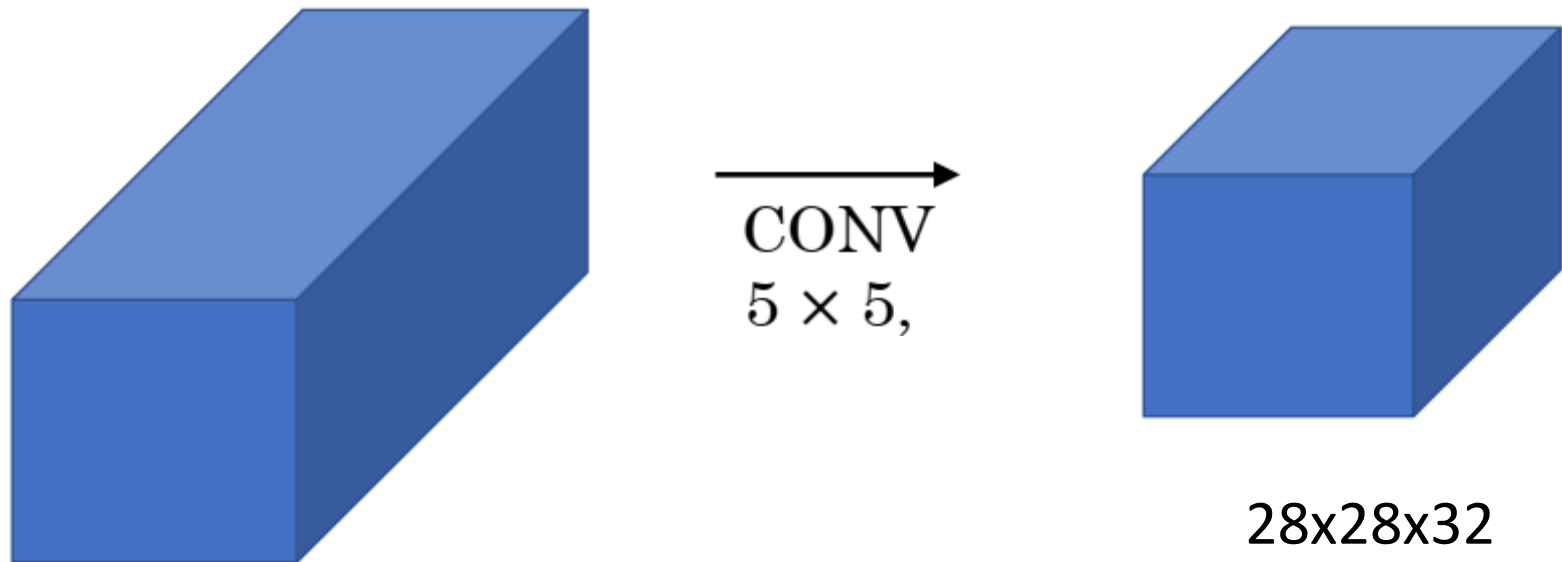


A number of filters goes from 192 to 32.

Inception Module



Computational Cost

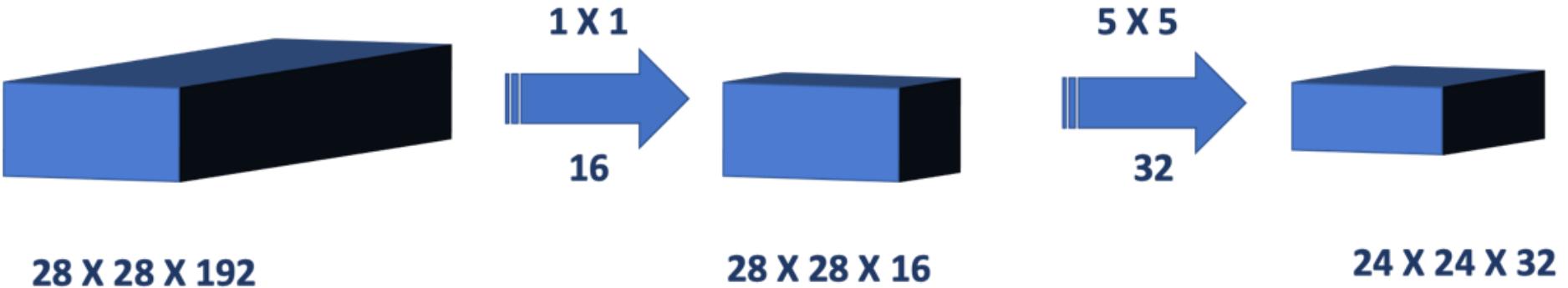


$28 \times 28 \times 192$

32 filters each having size $5 \times 5 \times 192$

Total number of ops = $28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120\text{M}$

High Computation Cost

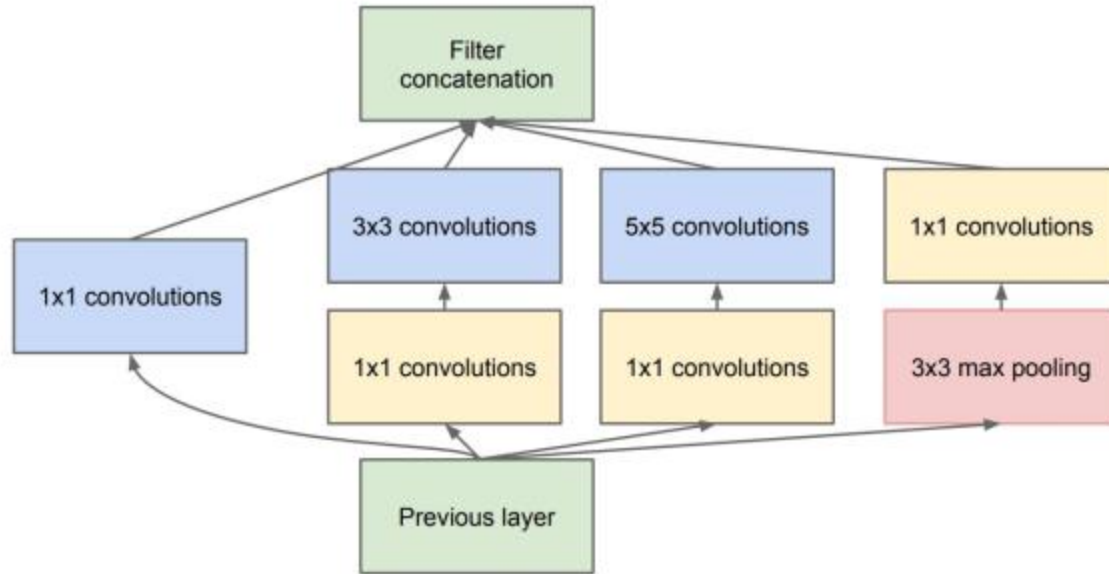


Number of Operations for 1 X 1 Conv Step : $(28 \times 28 \times 16) \times (1 \times 1 \times 192) = 2.4$ Million Ops

Number of Operations for 5 X 5 Conv Step : $(28 \times 28 \times 32) \times (5 \times 5 \times 16) = 10$ Million Ops

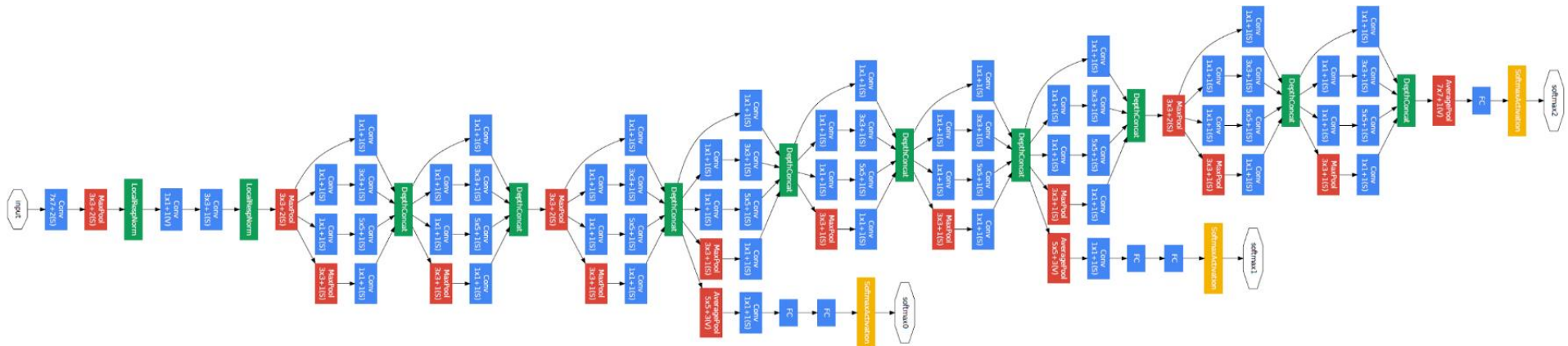
Total Number of Operations = 12.4 Million Ops

Inception Module with Dimension Reduction





Inception/GoogLe Network



Object Detection and Basic Object Detection Algorithms

Some of the Building Blocks for Object Detection

Courtesy: Andrew NG

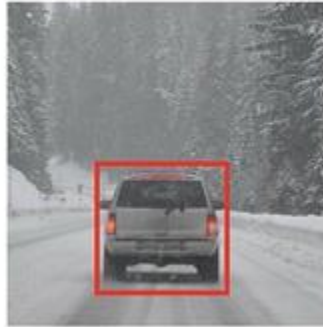
What is Object localization and Detection?

- Object Detection
- Object Localization

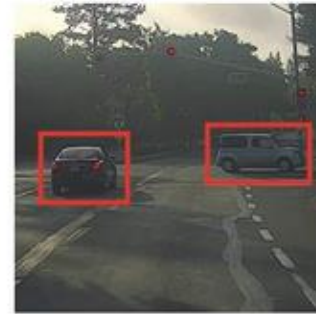
Image classification



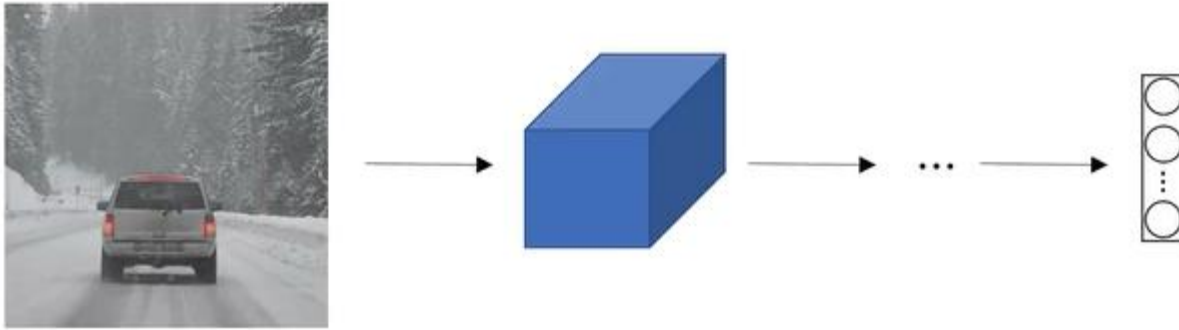
Classification with localization



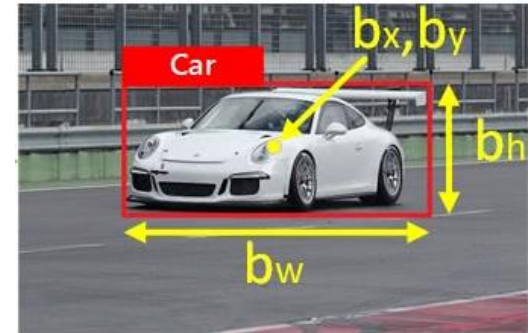
Detection



Classification with Localization



- Object categories for self-driving car
1. Car 2. Bike, 3. Truck, 4. Background
- Softmax with 4 possible outputs and
4 output units for bounding box (b_x, b_y, b_h, b_w)



How to define the target label Y

- 4 Object categories: 1. Car 2. Bike, 3. Truck, 4. Background
- Need to output bx, by, bh, bw, and class labels (1-4)

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$



$$\begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 0 \\ 0 \end{bmatrix}$$



$$\begin{bmatrix} 0 \\ \sim \\ \sim \\ \sim \\ \sim \\ \sim \\ \sim \\ \sim \end{bmatrix}$$

$$L(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 \\ + \dots + (\hat{y}_8 - y_8)^2 & \text{if } y_1 = 1 \\ (\hat{y}_1 - y_1)^2 & \text{if } y_1 = 0 \end{cases}$$

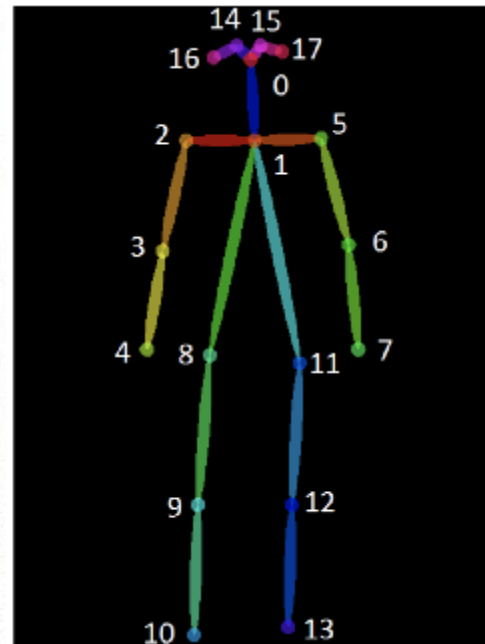
Landmark Detection



b_x, b_y, b_h, b_w

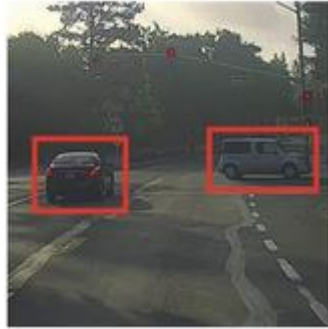


$l_{1x}, l_{1y},$
 $l_{2x}, l_{2y},$
 $l_{3x}, l_{3y},$
 $l_{4x}, l_{4y},$
 $\vdots$
 l_{64x}, l_{64y}



Object Detection

- Can we use Convolution Neural Net for Object Detection?
- Object Detection algorithm: Conv. Neural Net with sliding windows
- Car detection example



Training set:



→ ConvNet → y

Sliding windows Detection (for Object detection)

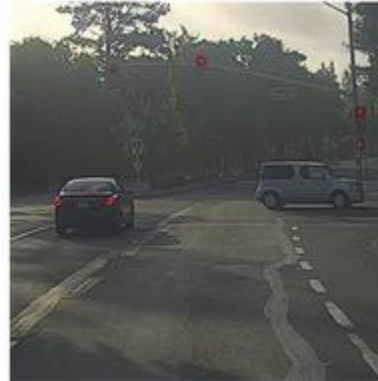
- The trained ConvNet can be used in sliding windows detection. (Slide window across the entire image)



Sliding windows Detection (for Object detection)

- Sliding window with Larger size

→ ConvNet → 0

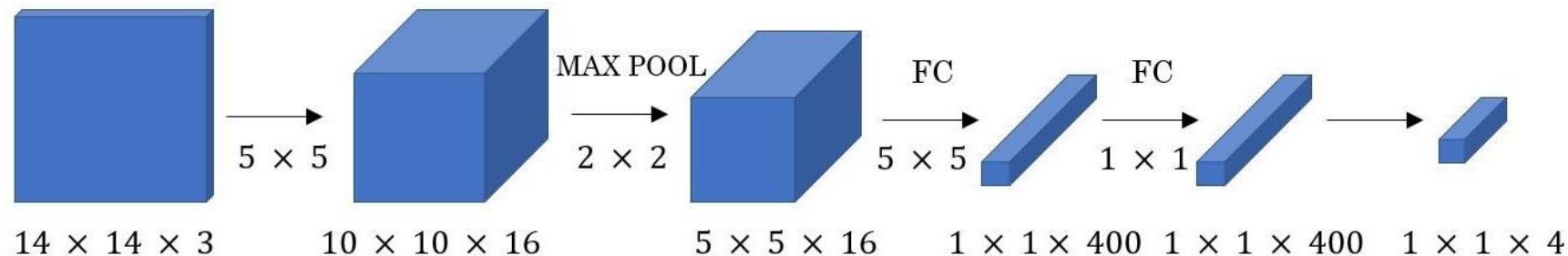
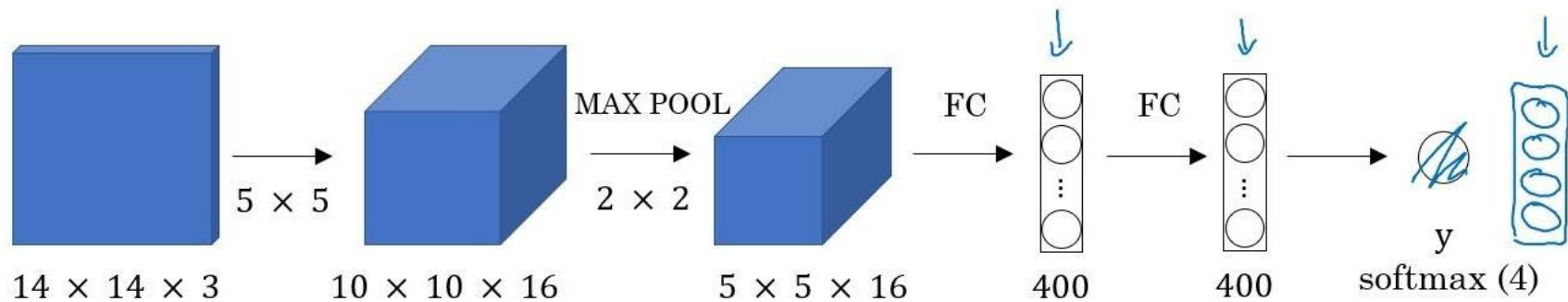


Convolution

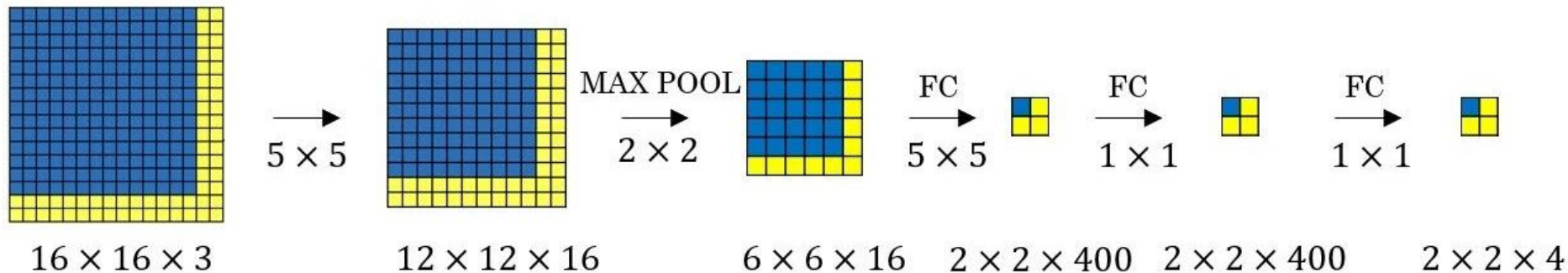
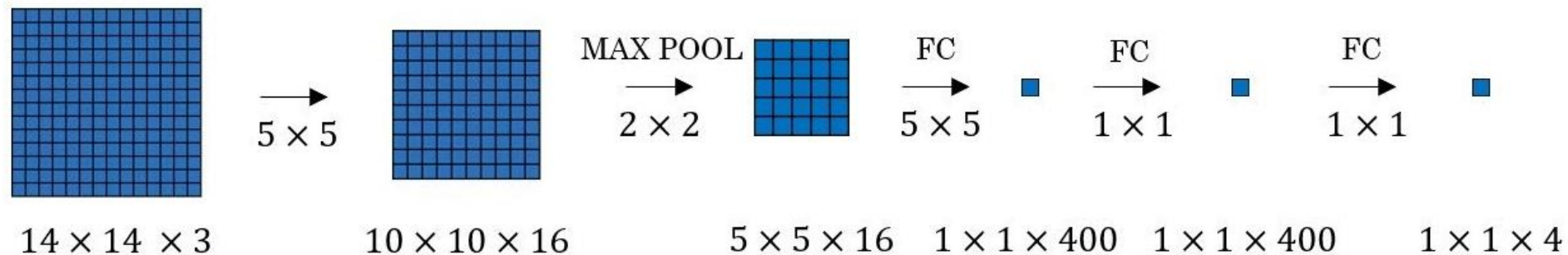
Implementation of Sliding Windows

Convolution Implementation of Sliding Windows

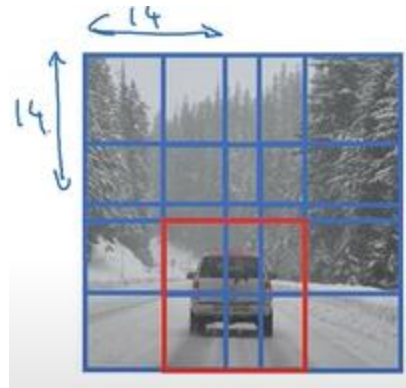
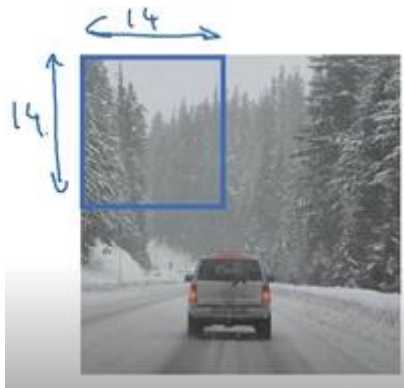
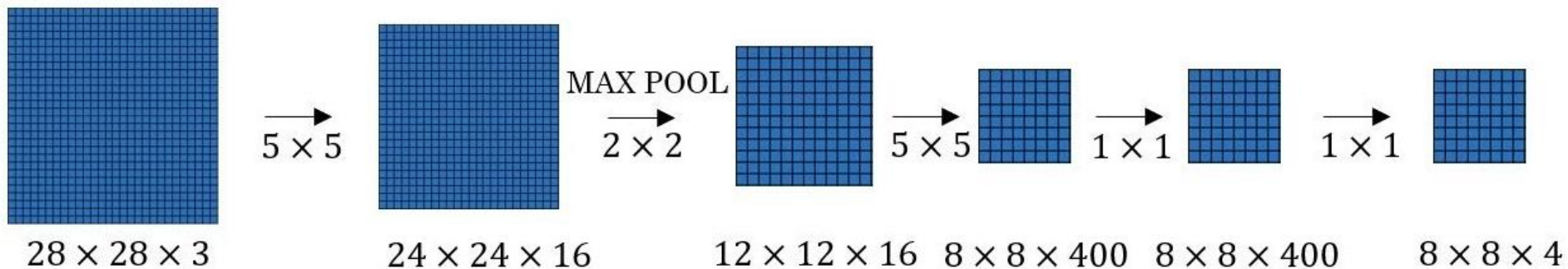
- Fully connected layers are converted in convolution layers



Convolution Implementation of Sliding Windows



Convolution Implementation of Sliding Windows



Convolution Implementation of Sliding Windows

- Weakness:
 - Bounding boxes may not be really accurate

